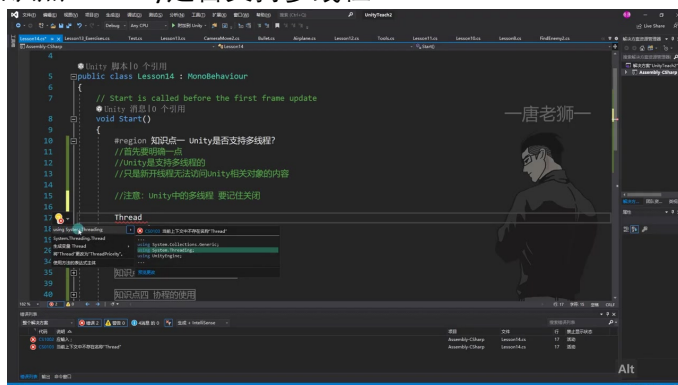


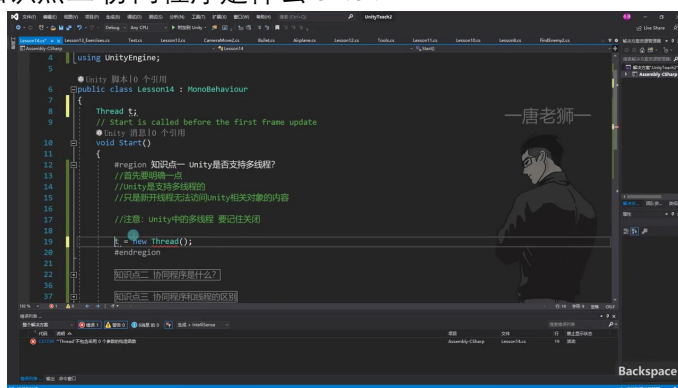
## 一、协同程序 00:04

### 1. 知识点一 Unity是否支持多线程 00:47



- 支持情况：Unity是支持多线程的，但新开的线程无法访问Unity相关对象的内容
- 注意事项：Unity中的多线程必须记住关闭，否则会造成资源泄漏
- 实现方法：
  - 使用C#的Thread类，需引用System.Threading命名空间
  - 线程应作为成员变量声明，便于后续管理
  - 线程构造函数需要传入委托函数作为参数
  - 通过Start()方法启动线程

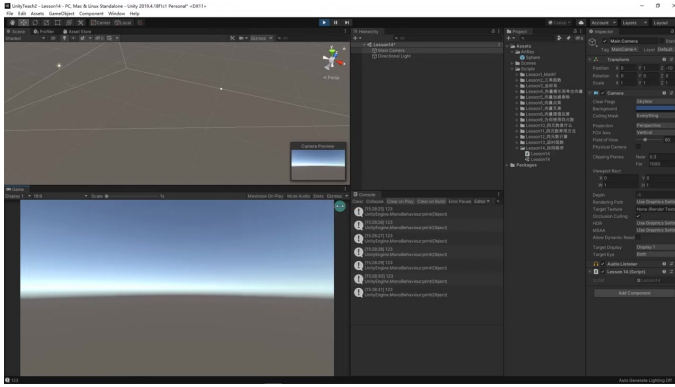
### 2. 知识点二 协同程序是什么 01:04



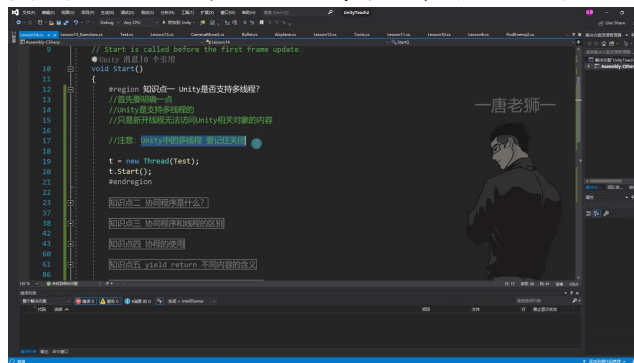
- 定义：协同程序简称协程，是一种"假"的多线程
- 本质：并非真正的多线程，而是在主线程上分时执行的逻辑
- 主要作用：
  - 将代码分时执行，避免主线程卡顿
  - 把耗时逻辑分步执行，提高程序响应性
- 使用场景：
  - 异步加载文件
  - 异步下载文件
  - 场景异步加载
  - 批量创建对象时防止卡顿

### 3. 知识点三 协同程序和线程的区别 01:56





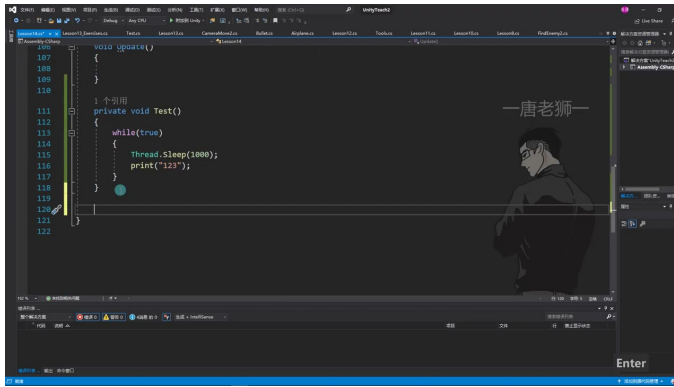
- 
- **线程死循环实现计时：**通过创建线程死循环并配合Thread.Sleep(1000)方法，可实现每隔1秒执行一次循环内容
- **打印效果展示：**线程会持续打印"123"，在控制台中表现为不断叠加的日志输出（展开后可见逐条记录）
- **编辑器线程特性：**
  - Unity编辑器环境下线程会持续运行，即使停止游戏也不会自动终止
  - 本质原因是线程与编辑器进程共生，只要Unity软件未关闭或脚本未修改，线程就会持续执行
- **重要注意事项：**
  - 必须手动关闭Unity中创建的多线程，否则会造成资源泄漏
  - 典型实现方式：在OnDestroy等生命周期函数中调用线程终止方法



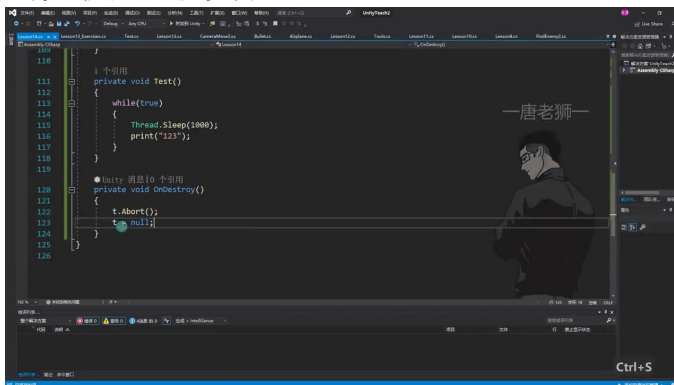
- 
- **多线程支持确认：**
  - Unity明确支持多线程编程
  - 限制条件：新开线程不能直接访问Unity相关对象和组件
- **线程管理规范：**
  - 创建：通过new Thread()实例化并调用Start()启动
  - 销毁：必须实现线程资源的主动释放机制
  - 典型错误：未关闭的线程会导致编辑器持续占用系统资源

## 6. 知识点六 04:32

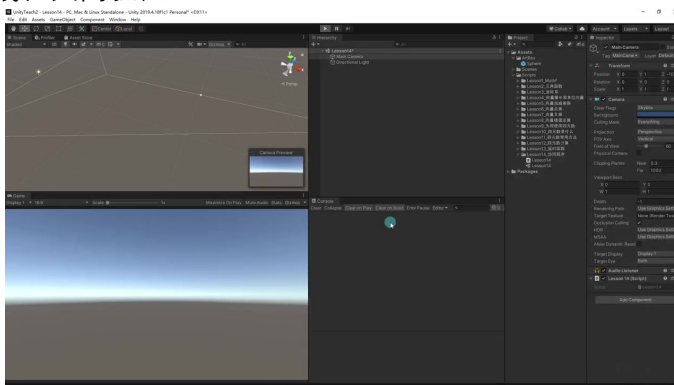
### 1) Unity多线程关闭方法



- - **生命周期函数选择:** 使用OnDestroy()生命周期函数来关闭线程，因为当对象移除或编辑器关闭时，该函数会自动执行。
  - **线程关闭方法:**
    - 通过Thread对象的Abort()方法来终止线程执行
    - 关闭后需要将线程对象置为null，完全释放资源占用
  - **关键代码示例:**
- 2) 多线程使用注意事项



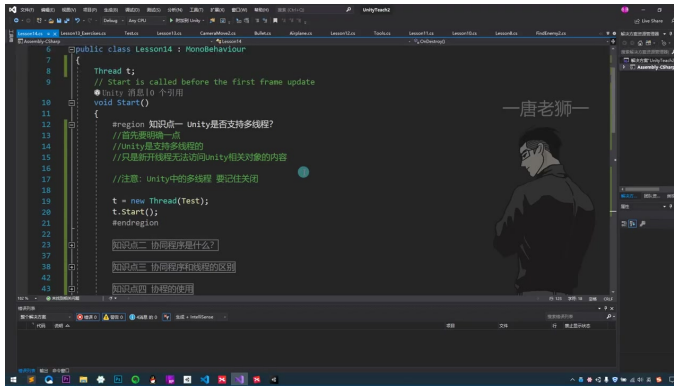
- - **必要性:** Unity中开启的多线程必须手动关闭，否则会持续占用系统资源
  - **资源占用:** 未关闭的线程会与编辑器共生，即使停止运行也会继续执行
  - **验证方法:** 通过打印日志可以观察到线程是否仍在运行（如示例中的"123"循环打印）
- 3) 线程关闭验证



- 
- **验证过程:**
  - 运行程序时线程会持续执行（如打印"123"）
  - 停止运行后，由于OnDestroy()的执行，线程会被正确终止
- **重要性:** 这种验证方式可以确保线程管理机制的有效性

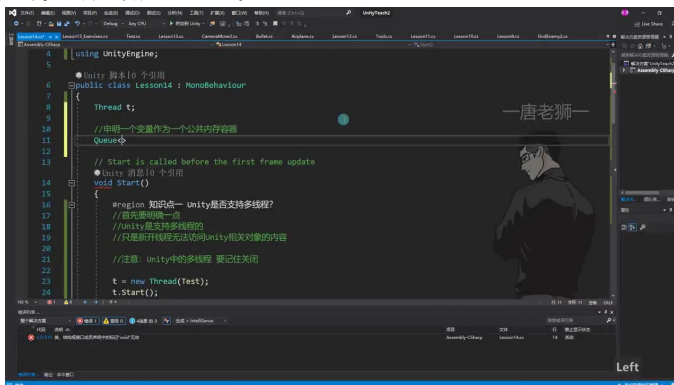
## 7. Unity多线程与协程 05:38

### 1) Unity多线程支持与限制 05:46



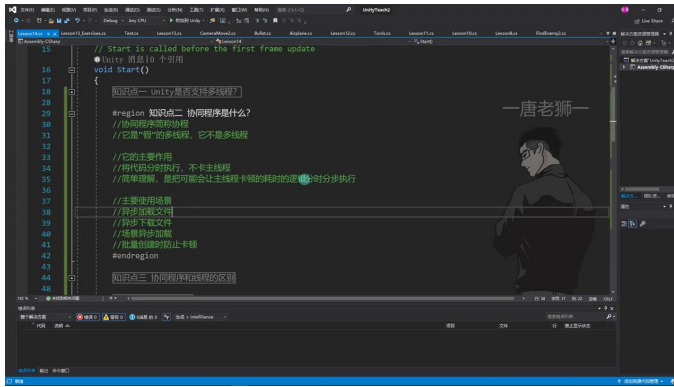
- 
- **支持情况：** Unity确实支持多线程，可以创建并运行新线程
- **关键限制：**
  - 新线程无法访问Unity相关对象的内容（如Transform、GameObject等）
  - 尝试访问会抛出"get\_transform can only be called from the main thread"异常
- **注意事项：**
  - 必须手动关闭创建的多线程（在OnDestroy中使用t.Abort()）
  - Unity大部分API都不能在非主线程中调用

## 2) 多线程数据交互方案 09:12



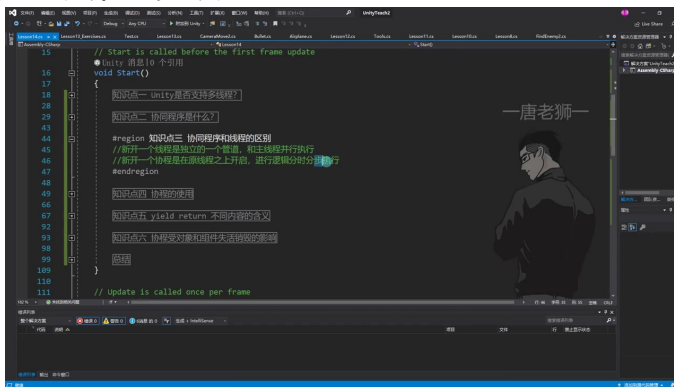
- 
- **交互原理：**
  - 使用共享容器（如Queue）作为线程间数据交换的中间媒介
  - 子线程负责计算，主线程负责使用计算结果
- **实现步骤：**
  - 声明公共队列： Queue<Vector3> queue = new Queue<Vector3>()
  - 子线程计算并存入： queue.Enqueue(计算结果)
  - 主线程检查并使用： if(queue.Count>0) transform.position = queue.Dequeue()
- **适用场景：**
  - 复杂算法计算（如A\*寻路）
  - 网络通信和下载
  - 批量对象创建

## 3) 协程基本概念 17:11



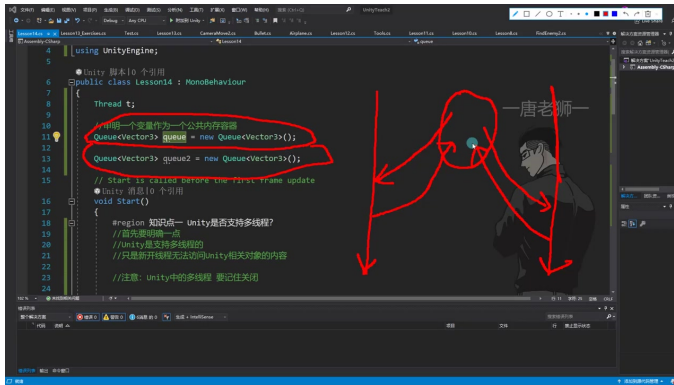
- 
- **本质:**
  - 不是真正的多线程，而是在主线程上分时分步执行的迭代器
  - 通过yield return控制执行流程
- **核心作用:**
  - 将耗时逻辑分步执行，避免主线程卡顿
- **典型应用:**
  - 异步加载资源
  - 场景异步切换
  - 批量对象创建时的性能优化

#### 4) 协程与线程的区别 18:24



- 
- **执行方式:**
  - 线程：独立管道，与主线程并行执行
  - 协程：在主线程上分时分步执行
- **执行控制:**
  - 线程：执行过程不可控
  - 协程：可通过yield精确控制每一步的执行时机
- **资源访问:**
  - 线程：不能直接访问Unity对象
  - 协程：可以自由访问所有Unity对象
- **性能开销:**
  - 线程：创建和切换开销较大
  - 协程：开销极小，适合高频使用

#### 5) 多线程使用建议



- 适用情况:
  - 需要执行复杂计算且不涉及Unity API
  - 需要长时间运行的后台任务（如网络通信）
- 注意事项:
  - 必须通过共享容器与主线程交互数据
  - 避免在子线程中使用任何Unity引擎API
  - 确保线程在对象销毁时正确关闭
- 替代方案:
  - 对于需要访问Unity对象的异步操作，优先考虑使用协程
  - 对于简单延时操作，使用Invoke方法更简便

## 二、知识小结

| 知识点        | 核心内容                                | 考试重点/易混淆点                 | 难度系数 |
|------------|-------------------------------------|---------------------------|------|
| Unity多线程支持 | 支持多线程但有限制：新线程无法访问Unity对象，主要用于复杂算法计算 | 主线程与副线程数据交互方式（通过公共容器）     | ★★★★ |
| 协同程序本质     | 分时分步执行的迭代器：非多线程，代码分段执行控制            | 与线程的本质区别（单线程内分步 vs 多线程并行） | ★★★  |
| 线程与携程对比    | 线程：独立管道并行执行<br>携程：主线程内逻辑分片执行        | 携程的挂起机制与执行条件控制            | ★★★★ |
| 多线程应用场景    | 算法计算/网络通信：A*寻路、网络消息收发通过公共队列交互数据     | 副线程禁止调用Unity API的限制       | ★★★★ |
| 携程核心作用     | 防止主线程卡顿：异步加载                        | yield return不同返回值的含义控制    | ★★★  |

|          |                                                    |                                                    |      |
|----------|----------------------------------------------------|----------------------------------------------------|------|
|          | /下载、批量<br>创建场景                                     |                                                    |      |
| 线程生命周期管理 | <b>必须手动关闭</b> ：通过<br>OnDestroy终止<br>线程，避免编<br>辑器共生 | 线程休眠<br>(Thread.Sleep)与主<br>线程Time.deltaTime<br>区别 | ★★★★ |